

Zadanie 1 Zakładam, że wierzchołków jest n i są ponumerowane liczbami od 1 do n oraz korzeń ma numer 1. Przygotowanie wygląda następująco: tworzę 4 tablice długości n - `PreOrder`, `PreOrderInv`, `PostOrder`, `PostOrderInv`. Tablice `PreOrder` i `PostOrder` tworzę przeglądając drzewo w odpowiednich porządkach. To wykonuje się w czasie $\Theta(n)$ i nie wymaga więcej pamięci niż $\Theta(n)$. Powstałe tablice tworzą dwie permutacje $\{1, \dots, n\}$ i w `PreOrderInv` i w `PostOrderInv` wpisuje ich odwrotności następująco. Iterując się po każdym indeksie $i = 1, \dots, n$ wpisuje: `PreOrderInv[PreOrder[i]] = i` oraz analogicznie dla drugiej tablicy: `PostOrderInv[PostOrder[i]] = i`. To wykonuje się w czasie $\Theta(n)$ i kończy przygotowania. Co sprawia, że pamięciowo zużywamy $4n \in \Theta(n)$ oraz czasowo podobnie co najwyżej $5n \in \Theta(n)$. Zatem całe przygotowania mają złożoność czasową $\Theta(n)$ i pamięciową $\Theta(n)$.

Teraz dostajemy dwa wierzchołki (które interpretuję jako liczby od 1 do n) - x i y . I chcemy określić, czy x jest przodkiem y . Jeśli $x = y$ to zwracamy odpowiedź pozytywną (jeśli dobrze rozumiem węzeł jest swoim własnym przodkiem). Jeśli $x \neq y$ to postępujemy następująco: sprawdzamy czy `PreOrderInv[x] < PreOrderInv[y]` i czy `PostOrderInv[x] > PostOrderInv[y]` - jeśli oba zdania były prawdziwe to zwracamy odpowiedź pozytywną, w przeciwnym wypadku - fałsz. Dla dowolnych dwóch x, y wykonujemy jedną lub dwie operacje, zatem to sprawdzenie ma złożoność czasową $O(1)$.

Aby udowodnić poprawność tego algorytmu, należy udowodnić następującą równoważność:

$x \neq y$ jest przodkiem $y \iff x$ występuje wcześniej niż y w porządku `PreOrder` oraz x występuje później niż y w Porządku `PostOrder`

“ $\implies$ ” Oczywiście podczas przeglądania drzewa porządkiem `PreOrder`, najpierw wypiszemy x , a potem y , który jest potomkiem x więc będzie wypisany w części rekursywnie odpowiadającej któremuś dziecku x . Analogicznie dla `PostOrder` x będzie wypisany *po* wszystkich swoich potomkach.

“ $\impliedby$ ” Rozważmy y , które jest później niż x w porządku `PreOrder`. Jeśli y nie jest potomkiem x to rozważmy najniższego wspólnego przodka x i y - oznaczmy go p . Podczas przeglądu drzewa w porządku `PreOrder` spotykając p wypisujemy go, a potem wszystkich “lewych” potomków, potem “prawych”. x musi być “lewym” potomkiem, y “prawym”, bo gdyby tak nie było, to p nie byłby najniższym wspólnym przodkiem x i y lub y byłoby przed x w porządku `PreOrder`. Rozważmy przegląd drzewa w porządku `PostOrder` - gdy napotkamy na p najpierw wypisujemy “lewych” potomków p , w tym x , a potem prawych potomków p , w tym y , ale tutaj mamy sprzeczność, bo y powinien pojawiać się *przed* x w porządku `PostOrder`, a pojawia się po nim.

Ta równoważność pokazuje poprawność algorytmu.